

MBR Intelligent Site Assistant

User Guide · Version 0.8.1

A self-hosted conversational site search for WordPress.

No external APIs. No monthly fees. Nothing leaves your server on the visitor path.

Published by Little Web Shack · littlewebshack.com

Author: Robert Palmer · [GPL-2.0-or-later](#)

Contents

1. Introduction
2. What's new since version 0.7.0
3. How it works
4. Installation
5. First-time setup
6. The diagnostic dashboard
7. The widget: placement and customisation
8. Intents — managing through the admin UI
9. Synonyms — managing through the admin UI
10. Appearance — themes and glass effect
11. Indexing behaviour
12. Command line (WP-CLI)
13. REST API
14. Privacy, data, and rate limiting
15. Troubleshooting
16. Uninstalling and keeping data
17. Technical reference
18. Support and version information

1. Introduction

The MBR Intelligent Site Assistant is a WordPress plugin that adds a conversational chat widget to your site. Visitors type questions in plain English, and the assistant responds with short messages and links to the most relevant pages on your site.

The plugin runs entirely on your own server. There is no external API call for each query, no monthly subscription, and no visitor data sent to a third party. The ranking is done in pure PHP and MySQL using a well-understood information retrieval algorithm (BM25), layered with intent matching for common questions and synonym expansion for natural phrasing.

1.1 What it does

- Adds a floating chat bubble, or an inline widget via shortcode.
- Indexes your published posts and pages automatically when they are saved — and any custom post types you tick in the admin.
- Indexes the text inside your PDF files (optional), so their contents are searchable alongside posts and pages.
- Splits long documents into overlapping passage chunks (new in 0.8.0), so a relevant passage deep inside a long PDF ranks on its own merits.
- Matches common questions (“contact”, “pricing”) to pre-written answers, with HTML support for links and basic formatting.
- Expands queries with synonyms so “WP” finds content tagged “WordPress”.
- Logs queries and thumbs-up / thumbs-down feedback so you can tune responses.
- Provides full admin UIs for intents, synonyms, and widget appearance — no code editing required.
- Five colour-coordinated themes plus an optional glassmorphism effect, all configurable from the admin.
- Ships WP-CLI commands (new in 0.8.1) for reindexing and status checks from the command line.

1.2 What it is not

Despite the word *assistant*, this plugin is not a large language model and does not generate text. It retrieves and surfaces content you have already written, and answers a configurable list of canned questions. If a visitor asks something you have not written about, the assistant will politely say it could not find anything, and invite them to rephrase.

This is a deliberate design choice. The plugin produces predictable, auditable responses with no hallucination risk — and it does it without sending your visitors’ questions to an outside service.

PDF indexing follows the same principle: it reads the text that is already inside your PDFs. It does not perform OCR on scanned images, and it does not decrypt protected files.

2. What's new since version 0.7.0

If you are upgrading from v0.7.0 (the previous published version of this guide), here is the headline summary. Each item is covered in detail in its own chapter.

2.1 Version 0.7.1 — PDF extraction and ranking fixes

- **Fixed:** PDF extraction could emit binary garbage into the index and into search snippets. Embedded font programs are compressed exactly like page content and can inflate to binary that happens to contain text-showing operators. Decoded streams must now look like text before they reach the parser, and the text-operator check requires the operator in showing position rather than as a raw substring.
- **Fixed:** Long documents were unfairly buried in results. BM25 length normalisation was applied per field but against the *total* document length, so a short title match on a long PDF inherited the length penalty of its entire body. Each field is now normalised against its own length and its own corpus average.
- **Added:** The Search Tester trace reports average field length per field.

2.2 Version 0.8.0 — passage chunking

- Long documents are split into overlapping passage chunks of roughly 250 words. Each chunk is indexed and ranked as its own BM25 unit, then collapsed back to one result per document at query time. See § 11.6.
- Search snippets are built from the passage that actually matched, windowed around the first highlighted term — so a snippet shows *why* a result matched rather than merely how the document begins.
- The Diagnostics page reports a chunks count alongside documents (which continues to mean distinct posts, pages, and PDFs).
- Two new settings keys, `chunk_size_words` and `chunk_overlap_words`, allow tuning. The defaults suit most sites. See § 17.1.
- Database schema v2. The upgrade runs automatically on the first admin page load after updating. A full reindex is required afterwards — see § 4.4.

2.3 Version 0.8.1 — WP-CLI commands

- `wp mbr -isa reindex` runs a full reindex from the command line, free of the web request's time and memory limits. This is the recommended route for large sites and large PDF libraries. See § 12.
- `wp mbr -isa status` prints plugin version, schema version, index counts, PDF indexing state, and the time of the last full reindex.
- The CLI class is loaded only when WP-CLI is running; ordinary web requests are unaffected.
- PDF extraction now strips table-of-contents leader runs and the page numbers they strand, so snippets are no longer filled with rows of dots. See § 11.5.
- Passages that look like a contents page are demoted in ranking, so the section that answers a question outranks the list of section titles. See § 11.6.

3. How it works

When a visitor submits a question, the plugin runs it through a six-stage pipeline. Understanding this pipeline will help you tune the plugin effectively.

3.1 The query pipeline

- **Intent match.** The raw query is checked against configured intent triggers. If any trigger matches as a substring (or regex, if prefixed with `re :`), the matching intent's canned response is returned immediately. Search is skipped. Disabled intents are ignored.
- **Tokenisation.** If no intent matched, the query is lowercased, stripped of punctuation, split on whitespace, filtered against an English stopword list, and each surviving word is reduced to its stem using a Porter stemmer.
- **Synonym expansion.** Each stemmed token is looked up in the synonym map. Matching terms add their siblings to the token list.
- **Ranking.** The expanded tokens are scored against the inverted index using BM25, with title matches weighted higher than body matches. Each field is normalised against its own length (0.7.1+), and each passage chunk is scored independently (0.8.0+).
- **Collapse.** Because a long document occupies several chunk rows, the ranked list is collapsed to the single best-scoring chunk per document, so a visitor never sees the same page listed twice.
- **Formatting.** Results are classified by confidence (high, medium, low, or none), a matching message is chosen, and a short highlighted snippet is built around the query terms within the winning passage.

3.2 Confidence levels

The responder picks one of four confidence levels based on the top result's BM25 score and how much it dominates the second result. Each level changes both the framing message and how many results are returned.

Level	Rule	Results	Typical framing
High	Top score ≥ 1.5 and $\geq 1.5\times$ the runner-up	1	"Here is what I found."
Medium	Top score ≥ 1.0	Up to 3	"These look relevant."
Low	Top score > 0 but below 1.0	Up to 3	"I'm not sure, but these might help."
None	No results	0	"I couldn't find anything."

3.3 The four database tables

The plugin creates four custom tables, all prefixed with your WordPress table prefix followed by `mbrisa_`:

Table	Purpose
<code>mbrisa_terms</code>	Dictionary of every unique token, with document frequency for BM25 IDF.
<code>mbrisa_documents</code>	One row per <i>passage chunk</i> . A short page is one row; a long PDF is many, distinguished by <code>chunk_index</code> . Each row stores title, URL, token count, content hash, and the passage text used for snippets.

Table	Purpose
mbrisa_postings	Inverted index — which terms appear in which chunks, with frequency per field (title / content / excerpt).
mbrisa_queries	Log of every query asked through the widget, with intent match, result count, top score, and feedback.

4. Installation

4.1 Requirements

- WordPress 5.8 or later
- PHP 7.4 or later
- Standard MySQL / MariaDB — no special extensions required for core search
- PDF indexing uses the standard zlib and mbstring extensions, which ship with virtually every PHP install
- No external services or API keys; no outbound network access is needed to run (only the optional update check reaches out — see § 4.3)

4.2 Installing the plugin

Because this plugin is distributed through Little Web Shack rather than the WordPress.org repository, installation is a manual upload:

- Download `mbr-intelligent-site-assistant.zip` from littlewebshack.com.
- In your WordPress admin, go to **Plugins** → **Add New**, then click **Upload Plugin**.
- Choose the ZIP file and click **Install Now**, then **Activate Plugin**.
- On activation, the four database tables are created and the default settings are seeded. Any existing published posts and pages will not be indexed automatically at this point — you need to run a full reindex once (see § 5).

***Note.** The floating chat widget is off by default on a fresh install. This is intentional — the plugin wants you to run a reindex and check the tokeniser before the widget starts talking to real visitors.*

4.3 Automatic updates

The plugin keeps itself up to date through the normal WordPress Plugins screen — no WordPress.org listing required. It periodically checks a public GitHub manifest for the latest version, and when you click update it pulls the package from littlewebshack.com, exactly like any other plugin update.

The update check is a server-side version lookup only. It transmits no visitor data, no site content, and nothing that identifies you or your visitors. If you would rather the plugin made no outbound requests of any kind, you can remove the bundled update checker and update manually instead; see § 14.3.

4.4 Upgrading to 0.8.x — what to expect

Version 0.8.0 introduces database schema v2: the documents table gains a `chunk_index` column, its unique key changes from `(post_id)` to `(post_id, chunk_index)`, and the stored passage text is widened so snippets can be built from it.

- The migration runs automatically the first time an admin page loads after updating. Nothing is deleted — existing rows are preserved and continue to work.
- **Run a full reindex afterwards.** Content indexed under 0.7.x remains as single-row documents; chunking only applies to content indexed under 0.8.0 or later.
- If any PDF previously showed garbled text in its snippet, the reindex also rebuilds that stored text, clearing the problem fixed in 0.7.1.
- On a large site, run the reindex from the command line — see § 12.

Note. *If you use a page cache or CDN, the widget's assets and any cached pages may need clearing to pick up front-end changes. The index itself is served through a POST endpoint and is never page-cached.*

5. First-time setup

After activation, open **MBR Site Assistant** → **Diagnostics** in your WordPress admin (look for the chat icon in the sidebar). The diagnostic page is the central hub — from there you can reindex, test queries, configure the widget, enable PDF indexing, and reach the three specialised admin pages (intents, synonyms, appearance) via quick-access buttons at the top. The typical first-run sequence is: reindex, test, configure widget, pick a theme, enable the widget.

5.1 Run a full reindex

Scroll to the **Index Status** section and click **Run Full Reindex**. This wipes the index tables and rebuilds them from every published post and page (or whichever post types you have enabled in the settings), plus your PDFs if PDF indexing is switched on. Indexing 100 posts typically takes a few seconds on a modest host.

When it finishes you should see non-zero values for **Documents indexed**, **Chunks**, **Unique terms in dictionary**, and **Total postings**.

5.2 Test with the chat tester

At the bottom of the diagnostic page, the **Chat Tester** runs the full query pipeline exactly as the public widget would. Type a question you expect the plugin to handle — for example, “how do I contact you” (which should match the built-in Contact intent), or a word you know appears on several pages of your site.

Expand the **Raw response payload** details to see the exact JSON the widget will receive. This is invaluable when diagnosing why a query is not producing the result you expected.

5.3 Configure and enable the widget

Scroll up to **Widget Settings**. The important fields are:

Setting	Purpose
Floating widget	Tick to show the chat bubble on every page. Leave off if you only want to embed the widget inline via shortcode.
Position	Bottom-right (default) or bottom-left.
Widget title	Header text in the chat panel. Default: “Site Assistant”.
Greeting message	First message the assistant shows when the panel opens.
Input placeholder	Ghost text in the input box.

Save your changes. If you ticked **Floating widget**, the bubble will now appear on the front end of your site.

5.4 Pick a theme (optional)

Click the **Appearance** → button at the top of the diagnostic page. The default theme is **Mocha** (Catppuccin dark purple), which is what previous versions of the plugin shipped. If a different palette suits your site better, choose one of the four other presets — the live preview shows what the widget will look like, and the glassmorphism toggle adds an optional translucent effect. See § 10 for full details.

5.5 Choose your content sources (optional)

The **Content Sources** panel on the Diagnostics page controls what gets indexed. Out of the box that is published posts and pages; tick any additional post types you want included — custom post types appear in the list automatically once they are registered.

If you also want the contents of your PDFs to be searchable, tick **Index PDF files** and set a maximum file size if the default 20 MB does not suit your content. Save, then run a full reindex so existing content is picked up. Content added or changed afterwards is indexed automatically. See § 6.10, § 11.2, and § 11.5 for detail.

6. The diagnostic dashboard

The Diagnostics page at **MBR Site Assistant** → **Diagnostics** is the operational hub of the plugin. Three quick-access buttons at the top take you to the specialised admin pages; the body of the diagnostic page contains everything else.

6.1 Quick-access buttons

- **Manage intents** → — opens the intents admin page (§ 8).
- **Manage synonyms** → — opens the synonyms admin page (§ 9).
- **Appearance** → — opens the theme and effects admin page (§ 10).

6.2 Plugin Status

Quick sanity check: plugin version, database schema version, server PHP version, and the REST endpoint URL the widget will post to. If you are troubleshooting a broken widget, confirm this URL returns a valid response when you POST to it.

6.3 Database Tables

A four-row table showing whether each of the plugin's custom tables exists. Under normal operation all four should show a green tick. If any table is missing, deactivate and reactivate the plugin to trigger the activation routine again.

6.4 Index Status

Shows document, chunk, term, and posting counts, plus the timestamp of the last full reindex. Also hosts the **Run Full Reindex** button. A reindex is safe to run at any time — it wipes and rebuilds the index tables from scratch. You should run one after:

- Bulk-importing or bulk-editing posts
- Changing the list of enabled post types
- Enabling PDF indexing or changing the maximum PDF size
- Upgrading to 0.8.0 or later (see § 4.4)
- Changing the chunk size or overlap settings (§ 11.6)
- Modifying the stopword list or tokeniser (unlikely in normal use)
- Restoring from a backup

***Note.** **Documents** counts distinct posts, pages, and PDFs. **Chunks** counts the passage rows those documents occupy — on a site with several long PDFs the chunk figure will be considerably higher, which is expected and healthy.*

6.5 Widget Settings

All of the visible widget configuration lives here. See § 5.3 for a field-by-field breakdown and § 7 for more on widget placement.

6.6 Content Sources

Controls what the assistant indexes beyond posts and pages. See § 6.10 for the full breakdown.

6.7 Feedback & Queries

Seven-day statistics — total queries, thumbs up, thumbs down, how many returned zero results, and how many hit an intent. Below that is a table of the 30 most recent queries. Rows tinted red are zero-result queries or queries that received a thumbs-down. These are the ones worth looking at first, because each one represents either a missing intent, a missing synonym, or a gap in your site's content.

6.8 Tokeniser Tester

Paste any text and see exactly what the tokeniser does to it: original, cleaned, split, after stopwords, after stemming. Mostly useful when debugging — for instance, if you are surprised that a particular word is not matching, tokenise the query and tokenise the page title and compare.

6.9 Search Tester & Chat Tester

The **Search Tester** (raw BM25) runs the search pipeline without intent matching or synonym expansion, and shows a full trace of which terms were looked up and their scores — useful for confirming the ranking itself is behaving, independent of the higher-level stages. The trace reports matched documents, top score, and average field length per field. The **Chat Tester** (full pipeline) runs exactly what the public widget runs, showing the conversational output with the confidence badge and the raw JSON payload. This is the primary tool for tuning intents and synonyms.

6.10 Content Sources in detail

The Content Sources panel is where you choose what the assistant indexes. It has three controls:

Control	Purpose
Post types	A checkbox for each registered public post type. Ticked types have their published content indexed. Defaults to Posts and Pages.
Index PDF files	When ticked, PDFs in the Media Library are indexed and become searchable. Off by default.
Maximum PDF size	PDFs larger than this (in MB, default 20) are skipped, to protect memory on shared hosting. Range 1–100.

After changing anything here, run a full reindex so the change is applied to existing content. Once PDF indexing is enabled, PDFs are indexed on upload, re-indexed if the underlying file is replaced, and removed from the index when deleted.

Two details about the post type list are worth understanding:

- **Only publicly-viewable post types are listed.** Search results are returned through a public REST endpoint that performs no capability check at query time, so anything indexed is discoverable by any visitor. Private and internal post types are therefore excluded from the list by design — they would also have no reachable permalink for a result to link to.
- **Unregistered types are preserved, not dropped.** If a post type is enabled but is not currently registered — typically because the plugin or theme that registered it has been deactivated — it is shown in a separate group marked (*not registered*) and stays ticked. That way, saving the form while a plugin is temporarily inactive cannot silently remove its content from your configuration. Untick it to remove it deliberately.

Note. Unticking every post type is a valid configuration — for example, a PDF-only search. It is treated as “index no post types”, not as a reset to the defaults.

7. The widget: placement and customisation

7.1 Floating bubble mode

With **Floating widget** ticked in settings, a small chat bubble appears on every page of the front end, pinned to the bottom-right or bottom-left. Clicking the bubble opens the chat panel. The widget is automatically skipped on feeds, embeds, the favicon endpoint, and the robots.txt endpoint.

7.2 Inline mode via shortcode

To embed the widget within the flow of a specific page — for example, a dedicated “Ask” page — use the shortcode:

```
[mbr_isa_chat]
```

The shortcode supports four optional attributes that override the global settings for that one embed:

Attribute	Effect
title	Overrides the widget title shown in the panel header.
greeting	Overrides the first assistant message.
placeholder	Overrides the input placeholder text.
height	Sets the panel height — any valid CSS unit, e.g. 500px, 80vh.

Example:

```
[mbr_isa_chat title="Ask us anything" greeting="What are you looking for?" height="600px"]
```

7.3 Running both at once

It is safe to leave the floating widget on and also place a shortcode on a specific page. Each instance gets its own unique ID and operates independently. Session storage is shared, so a conversation started in one instance persists if the visitor switches to the other.

7.4 Assets and conditional loading

The widget’s CSS and JavaScript are only enqueued on pages where they are actually needed: either because the floating widget is enabled site-wide, or because the current page contains the shortcode. Pages that use neither do not load the assets.

7.5 Theming

Every visual aspect of the widget is themeable from the Appearance admin page (§ 10). Both modes — floating and inline — pick up whichever preset and effects you have configured site-wide. There is no shortcode override for theme; it is intentionally a single setting per site.

8. Intents — managing through the admin UI

An intent pairs a list of trigger phrases with a canned response. When a query matches any trigger, the intent's response is returned immediately — no search, no ranking. Intents are the single biggest usability win in a site assistant, because they let you give a perfect answer to every common question.

Intents are managed through a dedicated admin page rather than by editing the option in code. Find it at **MBR Site Assistant** → **Intents**, or via the **Manage intents** → button on the diagnostic page.

8.1 The intents page layout

- **Test panel** — type a query and see which intent (if any) would match, with the response, label, and confidence.
- **Existing intents** — one card per intent, each containing its own form.
- **Add a new intent** — a blank card at the bottom for creating intents from scratch.
- **Reset to defaults** — replaces all intents with the shipped defaults (with a JS confirmation).

8.2 Built-in intents

The plugin ships with five default intents, used as a starting point on first install. Once you save any change, the option becomes your own and the defaults stop applying — but you can always reset to them via the button at the bottom of the page.

ID	Example triggers
contact	contact, email address, get in touch, how do I reach
pricing	how much, what do you charge, pricing, your rates, cost of
services	what services, what do you do, what do you offer, can you build
hours	opening hours, what hours, when are you open, business hours
help	need help, can you help, I am stuck, not working

8.3 Editing an intent

Each intent card has six fields:

- **ID** — lowercase, dashes, underscores. Used internally; appears in the diagnostic logs.
- **Label** — human-friendly name shown in test results and diagnostics.
- **Confidence** — a number between 0.0 and 1.0. When more than one intent matches, the higher confidence wins.
- **Triggers** — one trigger per line. Substring match by default; prefix with `re :` for regex.
- **Response** — the message shown to visitors. Basic HTML allowed (links, bold, italic, line breaks, lists). Sanitised on save.
- **Disabled** — when ticked, the intent is kept in the list but does not fire. Useful for A/B testing or temporarily silencing an intent.

Each card has its own **Save changes** button, so an in-progress edit to one intent can never lose another's edits. Each card also has a **Delete intent** button (with confirmation).

8.4 HTML in responses

Responses go through `wp_kses_post ( )` on save, which permits the same set of tags WordPress allows in post content — including links with `<a href>`, `<strong>`, `<em>`, `<br>`, paragraphs, and lists. Anything outside that whitelist is stripped at save time, so the stored response is always safe to render.

On the front end, intent responses are rendered with the HTML preserved. Search responses (which use a different code path) remain plain text, as they always have.

8.5 How matching works

- Triggers are matched case-insensitively as substrings of the raw query.
- A trigger prefixed with `re:` is treated as a regex (useful for word-boundary matches, e.g. `re:\bhours?\b`).
- If more than one intent matches, the one with the highest confidence value wins.
- Disabled intents are skipped during matching.
- Empty queries are never matched; neither are intents missing an ID, triggers, or a response.

8.6 Validation rules

When you save an intent, the admin enforces a few rules to prevent obvious mistakes:

- ID must be unique. Trying to save a new intent with the same ID as an existing one is rejected with a clear error.
- ID must consist only of lowercase letters, numbers, dashes, and underscores.
- At least one trigger is required.
- Each `re:` regex trigger must be a valid pattern. Invalid regexes are rejected on save with the offending trigger named.
- The response, after stripping HTML, cannot be empty.

***Tip.** Use the Recent queries list on the diagnostic page to find questions that returned zero results, then add triggers or intents to cover them.*

9. Synonyms — managing through the admin UI

A synonym group is a list of terms that should be considered equivalent for searching. If any term in a group appears in a query, the others are added to the search before ranking. Matching happens on stems, not raw words, so “shops” and “shop” are already equivalent before synonyms kick in.

Synonym groups are managed through a dedicated admin page. Find it at **MBR Site Assistant** → **Synonyms**, or via the **Manage synonyms** → button on the diagnostic page.

9.1 Built-in groups

The defaults cover the common web-industry vocabulary. Each row is one group:

Group	Terms
1	wp, wordpress
2	site, website, webpage
3	ecommerce, e-commerce, shop, store, online shop
4	price, pricing, cost, fee, rate, quote
5	contact, email, reach, get in touch
6	help, support, assistance
7	plugin, extension, add-on, add-on
8	theme, template, skin
9	build, develop, create, make
10	seo, search engine optimisation, search engine optimization

9.2 Editing a group

Each group card has a single textarea. Enter one term per line — multi-word phrases like “search engine optimisation” are fine on a single line. Both newline-separated and comma-separated input is accepted on save, so pasting from a thesaurus also works.

A group needs at least two unique terms after parsing. Single-term groups do nothing useful, so they are rejected on save with a clear error.

Each card has its own Save and Delete buttons, mirroring the intents page. The header preview shows the first five terms of the group as a code chip — handy when scanning a long list.

9.3 The test expansion panel

Type a query into the test panel and the page shows two rows of token chips:

- **Original tokens** — the words from your query after stemming.
- **Added by synonyms** — the extra tokens that synonym expansion contributed, highlighted in green.

Each chip shows the stem followed by the original word in lighter brackets when the two differ. So `websit` (`website`) reads naturally, while `post` (unchanged by stemming) shows as just `post`. Visitors never see the stemmed form anywhere — this is admin-side feedback only.

***Note.** The Porter stemmer collapses related words to a common root so the index can match them as one. That's why "website" stems to "websit" and "running" stems to "run" — it's how the search actually works internally. The bracketed original word is purely for readability in the test panel.*

9.4 When to add a synonym vs an intent

Add a **synonym** when the missing phrasing is something visitors say but your content uses a different word. "Delivery" and "shipping" might both refer to the same page. Synonyms make the search find what's already on your site.

Add an **intent** when the question has a canned answer that does not correspond to any single page on your site. "What are your opening hours?" might not have a dedicated page — but you still want a good answer.

10. Appearance — themes and glass effect

The chat-widget colour scheme is configurable through a dedicated admin page. Find it at **MBR Site Assistant** → **Appearance**, or via the **Appearance** → button on the diagnostic page.

The page has three areas: a live preview at the top, a colour-preset selector in the middle, and an effects toggle at the bottom. Changes to the controls update the preview immediately; clicking **Save appearance** commits them to the front end.

10.1 The live preview

The preview renders an actual chat-widget structure using the same CSS as the public widget, over an animated gradient background that slowly cycles through a purple-pink-magenta palette. The moving background is intentional — the glass effect (see § 10.3) only really shows itself when there is something behind the panel for it to blur.

Clicking a preset card or toggling the glass checkbox updates the preview without saving. You can browse presets freely; nothing is committed until you press **Save appearance**. The preview includes a sample link, so you can check link readability per theme before going live.

10.2 The five presets

Each preset is a coordinated palette that overrides every colour in the widget. The presets are:

Preset	Kind	Description
Mocha	Dark	Catppuccin Mocha — the original default. Deep purple panel with mauve and lavender accents. Suits dark and design-led sites.
Slate Light	Light	Clean neutral greys with indigo accent. The corporate-friendly default — works on almost any site.
Ocean	Light	Soft light blues with teal accent. Reads as friendly and calm; good for service businesses, healthcare, education.
Sunset	Light	Warm peach base with coral accents. Distinctive and warm; suits hospitality, food, lifestyle sites.
Forest	Dark	Deep green forest with sage accent. The dark counterpart to Mocha for sites that want dark but not purple.

Existing installs upgrading from before v0.6.0 will see no visual change — Mocha is the default, and the original Catppuccin Mocha CSS is preserved exactly under that preset name.

10.3 Glassmorphism

The Effects section has a single toggle: **Glassmorphism**. When enabled, the chat panel becomes translucent with a backdrop blur, the bubble button gets a frosted-glass treatment, and the conversation bubbles within the panel get a softer glass overlay.

Glass works in combination with any of the five presets. The exact look differs by preset:

- On dark presets (Mocha, Forest), glass produces a **smoked-glass** effect — dark and translucent.
- On light presets (Slate Light, Ocean, Sunset), glass produces a **frosted-glass** effect — bright and translucent.

Glass relies on the CSS `backdrop-filter` property, which has wide modern browser support (Chrome, Edge, Firefox, Safari on current versions). Older browsers fall back to a partially transparent panel without the blur — readable, just less fancy.

***Tip.** Glass looks best over rich backgrounds. On a white-on-white page the effect is subtle by design — there is nothing visually interesting behind the panel for the blur to act on. On hero images, gradients, or photo-heavy pages, the effect carries.*

10.4 Saving and applying

Clicking **Save appearance** writes both `theme_preset` and `theme_glass` into the existing `mbr_isa_settings` option, preserving every other widget setting (title, greeting, placeholder, position, etc.). The front-end widget picks up the new theme on the next page load — no cache flush required by the plugin, though your own page/CDN/browser caches may need clearing to reveal the change.

11. Indexing behaviour

11.1 What gets indexed

By default, published posts and pages are indexed. The indexer pulls the title, post content, and excerpt fields, tokenises them, and stores per-field term frequencies so that title matches can be weighted higher than body matches at query time. PDF files in the Media Library can be indexed too (§ 11.5), and long content of any kind is split into passage chunks (§ 11.6).

Only published content is indexed. Drafts, pending, private, scheduled, and trashed content are excluded, and an item is removed from the index if it leaves published status.

11.2 Choosing post types

The post types to index are chosen from the **Content Sources** panel at **MBR Site Assistant** → **Diagnostics** (§ 6.10). Every registered public post type appears as a checkbox, including custom post types registered by your theme or other plugins — they show up in the list automatically, with no code required. Tick the ones you want, save, and run a full reindex.

***Note.** Only publicly-viewable post types are listed, because search results are served through a public endpoint that performs no capability check. See § 6.10 for the reasoning and for how post types that are enabled but no longer registered are handled.*

The selection is stored in the `enabled_post_types` entry of the plugin settings option. If you prefer to set it programmatically — in a deployment script, or from WP-CLI — the option can still be written directly:

```
$s = get_option( 'mbr_isa_settings', [] );
$s['enabled_post_types'] = [ 'post', 'page', 'product' ];
update_option( 'mbr_isa_settings', $s );
// Then: wp mbr-isa reindex
```

11.3 Automatic re-indexing on save

The indexer hooks `save_post`, `deleted_post`, and `trashed_post`. Every time you save or publish a post of an enabled type, the indexer recomputes the content hash; if it has changed, the document's entry in the index is rebuilt. If the post is unpublished, trashed, or deleted, it is removed from the index. Revisions and autosaves are ignored.

When PDF indexing is enabled, attachments are tracked on `add_attachment`, `edit_attachment`, and `delete_attachment` as well. Because a PDF's file can change without its post fields changing, the content hash also factors in the file's size and modification time — so replacing a file re-indexes it.

***Note.** Because the number of chunks a document produces can change between saves, a changed document has all of its chunk rows deleted and rewritten rather than updated in place. Document frequencies for affected terms are recalculated as part of the same operation, so the index stays consistent.*

11.4 Field weighting

BM25 is applied per field with the following default weights:

Field	Default weight
Title	3.0

Field	Default weight
Excerpt	1.5
Content	1.0

Weights are stored in the plugin settings option as `field_weight_title`, `field_weight_excerpt`, and `field_weight_body`. The BM25 parameters `bm25_k1` (default 1.2) and `bm25_b` (default 0.75) are also stored there.

Since 0.7.1 each field is length-normalised against its own average rather than against the whole document's length. Before that change, a short title match on a long document inherited the length penalty of the entire body, which pushed long documents — large PDFs above all — unfairly down the results.

11.5 PDF indexing

When enabled under **Diagnostics** → **Content Sources** (§ 6.10), PDFs in the Media Library are treated as first-class searchable documents. For each PDF the indexer extracts the text layer, tokenises it through the same pipeline used for posts, and stores it in the same index. Search results for a PDF link straight to the file.

Extraction is entirely local and dependency-free — pure PHP using only the standard `zlib` extension. It understands the common ways PDF text is stored and compressed (FlateDecode, ASCII85, and ASCIIHex), and reconstructs readable text including word spacing in justified documents and accented or non-Latin characters. Where a PDF also carries Media Library metadata (title, caption, description, alt text), that is folded in as well.

The following limits are worth knowing:

- **Text layer required.** Born-digital PDFs (exported from word processors, report tools, and the like) with standard or embedded fonts extract reliably. PDFs built with heavily subsetted or unusual fonts that lack a Unicode mapping may yield partial text.
- **No OCR.** Scanned or image-only PDFs contain no text layer, only a picture of the page. No optical character recognition is performed — that would require external software, contrary to the self-hosted design. Such files fall back to whatever Media Library metadata exists.
- **Encrypted PDFs are skipped.** They are not decrypted or indexed.
- **Size cap.** Files larger than the configured maximum (default 20 MB) are skipped, to protect memory on shared hosting.

Two kinds of page furniture are cleaned during extraction so they do not pollute search results: **leader runs** — the rows of dots joining a contents-page heading to its page number — and the page numbers those runs strand. Both are visual devices rather than text; left in place they swallow most of a snippet's window and inflate the passage length that BM25 measures against.

***Note.** On a very large PDF library, the first full reindex extracts every file in one operation, which on a constrained host may approach PHP's time or memory limits. The cleanest answer is to run the reindex from the command line, where those limits do not apply — see § 12.*

11.6 Passage chunking

New in 0.8.0. Before this change, a document was one row in the index no matter how long it was. That works well for pages and posts, but poorly for a thirty-page PDF: a single relevant paragraph was diluted across thousands of unrelated words, and the snippet shown to the visitor was always taken from the

beginning of the document — rarely the part that actually matched.

The indexer now splits long content into overlapping **passage chunks**. Each chunk becomes its own row and its own BM25 scoring unit, so a relevant passage competes on its own merits rather than being averaged out across the whole file.

How it works

- Content is split into windows of roughly 250 words, with a 50-word overlap between consecutive windows. The overlap means a sentence that straddles a boundary is present whole in at least one chunk, so a query matching it cannot fall down the crack.
- Content short enough to fit comfortably in one window (up to about 375 words) stays a single chunk. Ordinary pages and posts are therefore unaffected.
- The title is indexed with every chunk, so any matching passage still benefits from title relevance. The WordPress excerpt is indexed with the first chunk only.
- A final short remainder is folded into the preceding chunk rather than being emitted as a tiny fragment, which BM25's length normalisation would otherwise over-reward.
- At query time, the ranked list is collapsed to the best-scoring chunk per document, so a visitor never sees the same page or PDF listed twice.
- The winning chunk's text is what the snippet is built from, and the snippet is windowed around the first matching term — so the visitor sees the passage that caused the match.

Tuning

The defaults suit most sites and there is no admin UI for them, deliberately — they are the kind of setting that is better left alone than fiddled with. If you do need to change them, they live in the plugin settings option:

```
$s = get_option( 'mbr_isa_settings', [] );
$s['chunk_size_words'] = 250; // default
$s['chunk_overlap_words'] = 50; // default
update_option( 'mbr_isa_settings', $s );
// Then: wp mbr-isa reindex
```

- **Smaller chunks** (150–200 words) sharpen precision on documents that cover many unrelated topics, at the cost of more index rows.
- **Larger chunks** (350–500 words) keep more context together, which suits documents with long continuous arguments.
- Overlap is capped at a quarter of the chunk size if you set it too high.
- A change to either value requires a full reindex to take effect.

Contents pages

A table of contents lists every heading in a document, which makes it the densest concentration of topic vocabulary in the file — and a poor answer to any question, since the visitor gets section titles rather than prose. Left alone, a contents chunk frequently out-scores the passage that actually answers the query.

Passages that look like a contents page are therefore **demoted rather than excluded**: their score is multiplied by 0.4, so they surface only when nothing better matches. A visitor searching for a section title by name can still find it.

- Detection is structural: a run of ascending numbered entries (1 . , 2 . , 3) carrying very little connective prose.

- Numbered instructions inside a chapter are *not* affected — they read as prose within each entry, which is what the test measures.
- Data and reference tables are not affected either: decimals such as 1 . 5 or 0 . 75 are not counted as entry markers.
- A passage that mixes a short contents list with real prose is left alone, because it does contain content worth returning.
- The multiplier is tunable via `contents_score_penalty` in the settings option (default 0.4). Set it to 1.0 to disable the behaviour entirely.

Note. *Chunking increases the number of rows in the documents and postings tables — a thirty-page PDF might occupy forty chunk rows. This is expected. The **Documents** figure on the Diagnostics page still counts distinct posts, pages, and PDFs; **Chunks** counts the passage rows.*

12. Command line (WP-CLI)

New in 0.8.1. If your host provides SSH access with WP-CLI installed — most quality hosts do, including SiteGround, Kinsta, and WP Engine — the plugin exposes two commands under the `mbr -isa` namespace.

The commands are registered only when WP-CLI is running. Ordinary web requests never load the class, so there is no front-end cost to their existence.

12.1 wp mbr-isa reindex

Runs exactly the same full reindex as the button on the Diagnostics page — wiping the index tables and rebuilding them from every published item of the enabled post types, plus PDFs if PDF indexing is switched on.

```
$ wp mbr-isa reindex
Running full reindex...
Success: Indexed 32 document(s) as 118 chunk(s) in 12.482s.
Terms: 3038. Postings: 11370.
```

The advantage over the admin button is that CLI processes are not bound by the web server's `max_execution_time` or the PHP-FPM request memory ceiling. On a site with a large PDF library, this is the difference between a reindex that completes and one that dies halfway through with a gateway timeout.

Tip. This is also the tidiest way to perform the reindex required after upgrading to 0.8.x (§ 4.4), and the natural last step of a deployment script.

12.2 wp mbr-isa status

Prints a summary of the current index without changing anything — useful as a post-deployment health check.

```
$ wp mbr-isa status
Plugin version: 0.8.1
DB schema:      2
Documents:      32
Chunks:         118
Terms:          3038
Postings:       11370
PDF indexing:   on
Last full:      2026-08-03 09:14:22
```

12.3 Multisite and remote use

Both commands accept the standard WP-CLI global flags, so they work with `--url` on multisite and with aliases for remote servers:

```
$ wp mbr-isa reindex --url=https://example.com/site-two
$ wp @production mbr-isa status
```

12.4 Scheduling a periodic reindex

The plugin does not schedule anything itself — content is re-indexed as it is saved, so a routine full reindex is not normally required. If you want one anyway (after a nightly content import, for example), a cron entry is

the simplest approach:

```
# Rebuild the search index at 04:15 every day  
15 4 * * * cd /path/to/wordpress && wp mbr-isa reindex --quiet
```

Note. *There is deliberately no command to clear the index without rebuilding it. An empty index with the widget live means every visitor gets “I couldn’t find anything”, which is never what you want — and a reindex already starts by wiping the tables.*

13. REST API

The chat widget talks to the plugin via two REST endpoints, both under the `mbr-isa/v1` namespace. Both are public (no authentication) but nonce-protected when called from the widget, and rate-limited per IP.

Note. If you have a security or REST-hardening plugin installed (such as MBR WP Performance), make sure `mbr-isa/v1` is added to its REST allowlist. Otherwise the plugin will block the widget's requests and visitors will see a generic error message. See § 15 for the troubleshooting walk-through.

13.1 POST `/wp-json/mbr-isa/v1/ask`

Accepts a user question and returns the full structured chat response.

Request body:

```
{
  "query": "how do I contact you",
  "session_id": "optional-32-char-session-id"
}
```

Successful response (intent hit):

```
{
  "type": "intent",
  "intent_id": "contact",
  "intent_label": "Contact",
  "message": "You can get in touch via the contact form...",
  "message_html": "You can get in touch via the <a href=...>contact form</a>...",
  "message_format": "html",
  "results": [],
  "suggestions": ["Ask me something else about this site"],
  "session_id": "...",
  "query_id": 1234
}
```

Intent responses include `message_html` (the sanitised HTML response, rendered by the widget via `innerHTML`) and a `message_format` hint of `html`. The plain `message` field is the HTML-stripped version, kept as a safe fallback for any consumer that cannot render HTML. Search responses remain plain text.

Successful response (search hit, medium confidence):

```
{
  "type": "search_results",
  "confidence": "medium",
  "message": "These look relevant:",
  "results": [
    {
      "title": "...",
      "url": "...",
      "snippet": "... <mark>match</mark> ...",
      "score": 2.14
    }
  ],
  "suggestions": [],
  "session_id": "...",
  "query_id": 1235
}
```

Note. Result objects describe documents, not chunks — chunk collapse happens before the response is built, so each result is a distinct page or PDF. The snippet is drawn from whichever passage matched.

13.2 POST /wp-json/mbr-isa/v1/feedback

Records a thumbs-up (1), thumbs-down (-1), or neutral (0) against a previously-logged query.

```
{
  "query_id": 1234,
  "feedback": 1
}
```

Feedback is only accepted within one hour of the original query. Older queries return a 410 Gone response to discourage scripted bulk-feedback.

13.3 Error responses

Code	Error	Meaning
400	mbr_isa_invalid_query_id	query_id missing or not a positive integer
400	mbr_isa_invalid_feedback	feedback value not in {-1, 0, 1}
404	mbr_isa_not_found	query_id does not exist in the log
410	mbr_isa_feedback_expired	More than an hour has passed since the query
429	mbr_isa_rate_limited	Per-IP rate limit exceeded
500	mbr_isa_db_error	Database write failed when storing feedback

14. Privacy, data, and rate limiting

14.1 What the plugin stores

- **Post content, as an inverted index.** Stored across the `mbrisa_terms`, `mbrisa_documents`, and `mbrisa_postings` tables. This is derived from content you have already published.
- **Extracted PDF text** (when PDF indexing is enabled). Derived from files already in your Media Library, stored in the same index tables.
- **Passage text.** Each chunk row stores the passage it was built from, so snippets can be generated without re-reading the source file. This is your own published content, held in your own database.
- **Query log.** Each query submitted through the widget is logged in `mbrisa_queries` with the query text, matched intent (if any), top score, result count, optional feedback value, session ID, created-at timestamp, and a hashed version of the requester's IP address.
- **No personal identifiers.** The plugin does not record user accounts, cookies, emails, or raw IPs.

14.2 IP hashing

Where an IP address is needed — for rate limiting and for a loose approximation of unique askers in the query log — it is immediately hashed with SHA-256 and your site's WordPress salt. The raw IP is never stored. The hashed value is not reversible without access to your site's salt.

14.3 Outbound traffic

On the visitor path, the plugin makes **zero** outbound HTTP requests. No query — from the widget, the REST API, or anywhere else a visitor can reach — ever leaves your server. Every question is answered from data on your own server. This is a core design constraint, not an optional setting. PDF text is extracted locally too, in pure PHP, with no cloud OCR or extraction service. Ranking, chunking, and snippet generation are likewise entirely local.

The one exception is the update checker (§ 4.3), which performs a periodic server-side version check against a public GitHub manifest and downloads the package from `littlewebshack.com` when you choose to update. It transmits no visitor data, no site content, and no identifiers — it is a version-number check, the same as any WordPress plugin update. If you prefer no outbound requests of any kind, remove the bundled update checker and update the plugin manually.

14.4 Rate limiting

Both REST endpoints are protected by a per-IP-hash sliding window stored in WordPress transients. Defaults:

Endpoint	Default limit	Configurable via
<code>/mbr-isa/v1/ask</code>	30 requests / minute	<code>rate_limit_per_min</code> in settings
<code>/mbr-isa/v1/feedback</code>	20 requests / minute	Hardcoded for this release

When the limit is hit the endpoint returns 429 Too Many Requests and the widget shows a “you are asking too quickly” message.

15. Troubleshooting

The widget does not appear on the site

- Check that **Floating widget** is enabled under Widget Settings.
- Confirm your active theme calls `wp_footer()`. If it does not, the floating widget cannot render — use the shortcode instead.
- Open your browser's developer console and look for JavaScript errors from `chat-widget.js`.
- Confirm `window.mbrAisa` is defined with a `restUrl`. If not, something has stripped the localized script tag.

Widget shows “Something went wrong” on every query (401 from REST)

This is almost always caused by a security or performance plugin that disables the WordPress REST API for non-admin or logged-out users. Because the chat widget runs on the front end where most visitors are not logged in, a blanket REST blocker will catch every request.

- Check your dev tools Network tab. If the POST to `/wp-json/mbr-isa/v1/ask` returns 401 Unauthorized, this is the cause.
- If you have a REST-hardening plugin, look for an “allowed namespaces” or “REST whitelist” setting and add `mbr-isa/v1` to it. (MBR WP Performance v1.9.3 and later have this setting under Core → Allowed REST Namespaces.)
- If your plugin doesn't offer a whitelist, switch its REST mode to “Default (Enabled)” — that's the blunt fix and is fine if you trust the plugins on your site.

Queries return no results even though the content exists

- Run a full reindex. This is the most common cause on a fresh install — and is required after upgrading to 0.8.x (§ 4.4).
- Check Index Status shows a non-zero document count.
- Use the Tokeniser Tester on both your query and the page title — see which tokens actually come out.
- If the query uses different vocabulary to your content, add a synonym group via § 9.
- Confirm the post type is ticked under Diagnostics → Content Sources (§ 6.10).

A PDF isn't showing up in results

- Confirm **Index PDF files** is ticked under Diagnostics → Content Sources, and that you have run a full reindex since enabling it.
- Check the PDF is under the configured maximum size (default 20 MB).
- Remember that scanned / image-only PDFs have no text layer to index — only their Media Library metadata (title, caption, description, alt) is searchable. Add descriptive metadata to make such files findable.
- Encrypted PDFs are skipped by design.
- If the PDF is very long and the query is a common phrase, check the Search Tester: before 0.8.0, long documents were penalised by length normalisation. Upgrading and reindexing resolves this.

A PDF snippet shows unreadable characters

This was a bug in 0.7.0, fixed in 0.7.1: embedded font programs inside the PDF could be mistaken for page content and parsed as text. Update to 0.7.1 or later and run a full reindex — the stored snippet text is rebuilt from scratch, which clears the problem. Clearing caches alone will not fix it, because the garbled text is

stored in the database, not cached.

A search result is full of dots, or shows a list of section titles

Both symptoms come from a PDF's contents page winning the match. Update to 0.8.1 or later and run a full reindex: leader dots and stranded page numbers are removed during extraction, and contents-like passages are demoted so the chapter that answers the question ranks above the list of chapter names. If a contents page still wins on a particular query, lower `contents_score_penalty` (§ 17.1) below its 0.4 default.

A custom post type isn't being indexed

- Open Diagnostics → Content Sources and confirm the post type is ticked, then run a full reindex — ticking a type does not retrospectively index existing content on its own.
- If the post type isn't in the list at all, it is either not registered (check the plugin or theme that provides it is active) or not publicly viewable. Types registered with `'public' => false` are excluded by design — see § 6.10.
- Only published content is indexed. Content in draft, pending, private, or scheduled status will not appear.
- If the type is shown in the (*not registered*) group, whatever registers it is currently inactive; reactivate it and reindex.

An intent is firing when it should not

- Triggers are substrings, so a trigger like `he lp` will match “I need help finding the shop”. Either remove the trigger or tighten it with a regex prefix (`re:\bhe lp\b`).
- Use the Chat Tester to see which intent ID is firing.
- Consider toggling the **Disabled** checkbox on the intent rather than deleting it — that way you can re-enable later without re-typing the triggers.

Synonyms admin shows “websit” instead of “website” — is something broken?

No, that's the Porter stemmer doing its job. The search index uses the stemmed form internally so that “websites” matches “website” matches “webpages”. The admin test panel shows the bracketed original word alongside (e.g. `websit (website)`) precisely so it doesn't read as a typo. Visitors never see stemmed forms anywhere.

The widget says “too many requests”

- You are hitting the per-IP rate limit. Wait a minute and try again.
- If you are testing programmatically, raise `rate_limit_per_min` in the plugin settings option.

Reindex takes a long time or times out

- **Run it from the command line instead:** `wp mbr-isa reindex` (§ 12). CLI processes are not bound by the web server's execution time or request memory limits, which makes this the reliable answer on large sites.
- If CLI is not available, temporarily raise `max_execution_time` in `php.ini`.
- The reindex is synchronous and batches 50 posts (and 25 PDFs) at a time.
- PDF extraction is heavier than reading post fields — if a large PDF library is the bottleneck, reindex during a quiet period.

The index has far more chunks than documents

That is expected from 0.8.0 onwards. Long documents occupy several passage rows each (§ 11.6) — a thirty-page PDF might account for forty chunks on its own. The **Documents** figure is the one that

corresponds to pages and files on your site.

Glass effect doesn't show on the front end

- Confirm the toggle is on in MBR Site Assistant → Appearance.
- Glass requires `backdrop-filter` support in the visitor's browser. Modern Chrome, Edge, Firefox, and Safari all support it; very old browsers fall back to a partially transparent panel without blur.
- Glass is most visible over rich backgrounds (hero images, gradients). On a plain white page it is subtle by design.

A database table is missing after activation

- Deactivate and reactivate the plugin to re-run the activator.
- Check that the WordPress database user has `CREATE TABLE` permission on your database.

The schema didn't upgrade after updating to 0.8.x

- The migration runs on the first admin page load after the update. Visit any wp-admin page while logged in as an administrator.
- Check **Diagnostics** → **Plugin Status**: the DB schema version should read 2.
- If it still reads 1, deactivate and reactivate the plugin, which runs the activator directly.
- On a host with OPcache and `validate_timestamps=0`, flush OPcache after uploading plugin files — otherwise the old code keeps running and no migration is attempted.

16. Uninstalling and keeping data

Deactivating the plugin leaves all data intact — the tables, options, and widget settings are preserved so you can reactivate later without losing anything. Deleting the plugin (via Plugins → Delete) by default drops all four custom tables, all plugin options, and all transients.

16.1 Keeping data across a reinstall

If you plan to delete the plugin and reinstall later, set the `mbr_isa_keep_data_on_uninstall` option to a truthy value before deletion. With that option set, the uninstall routine skips all cleanup and leaves your index, query log, intents, synonyms, and theme settings untouched.

```
update_option( 'mbr_isa_keep_data_on_uninstall', 1 );
```


17. Technical reference

17.1 Plugin settings option

All widget, engine, and theme configuration lives in one option: `mbr_isa_settings`. The structure and defaults:

Key	Default	Purpose
<code>enabled_post_types</code>	<code>['post', 'page']</code>	Which post types are indexed. Set from Content Sources.
<code>bm25_k1</code>	<code>1.2</code>	BM25 term-frequency saturation constant.
<code>bm25_b</code>	<code>0.75</code>	BM25 length-normalisation constant.
<code>field_weight_title</code>	<code>3.0</code>	Score multiplier for title-field matches.
<code>field_weight_excerpt</code>	<code>1.5</code>	Score multiplier for excerpt matches.
<code>field_weight_body</code>	<code>1.0</code>	Score multiplier for content matches.
<code>chunk_size_words</code>	<code>250</code>	Target passage chunk size in words. (v0.8.0+)
<code>chunk_overlap_words</code>	<code>50</code>	Words shared between consecutive chunks. Capped at a quarter of the chunk size. (v0.8.0+)
<code>contents_score_penalty</code>	<code>0.4</code>	Score multiplier applied to passages that look like a contents page. 1.0 disables the demotion. (v0.8.1+)
<code>index_pdfs</code>	<code>false</code>	Index PDF attachments from the Media Library.
<code>pdf_max_filesize_mb</code>	<code>20</code>	Skip PDFs larger than this many MB.
<code>widget_enabled</code>	<code>false</code>	Show floating bubble site-wide.
<code>widget_position</code>	<code>'bottom-right'</code>	'bottom-right' or 'bottom-left'.
<code>widget_title</code>	<code>(empty)</code>	Overrides panel title if set.
<code>widget_greeting</code>	<code>(empty)</code>	Overrides greeting message if set.
<code>widget_placeholder</code>	<code>(empty)</code>	Overrides input placeholder if set.
<code>theme_preset</code>	<code>'mocha'</code>	One of: mocha, slate-light, ocean, sunset, forest.
<code>theme_glass</code>	<code>false</code>	Enable glassmorphism effect on the widget.
<code>log_queries</code>	<code>true</code>	Whether to log queries.
<code>rate_limit_per_min</code>	<code>30</code>	Max /ask requests per minute, per IP.

17.2 Other options

Option	Purpose
<code>mbr_isa_version</code>	Installed plugin version string.
<code>mbr_isa_db_version</code>	Schema version. 2 from v0.8.0 onwards.

Option	Purpose
<code>mbr_isa_intents</code>	Array of intent definitions (defaults used if empty).
<code>mbr_isa_synonyms</code>	Array of synonym groups (defaults used if empty).
<code>mbr_isa_index_status</code>	Cached counts shown on the diagnostic page: documents, chunks, terms, postings, and last full index time.
<code>mbr_isa_keep_data_on_uninstall</code>	If truthy, skip all cleanup on delete.

17.3 Intent data shape

Each entry in `mbr_isa_intents` is an associative array:

```
[
  'id'      => 'contact',
  'label'   => 'Contact',
  'triggers' => [ 'contact', 'email', 'get in touch' ],
  'response' => 'You can reach me at <a href="...">hello@example.com</a>.',
  'confidence' => 1.0,
  'disabled' => false,
]
```

17.4 Database schema notes

Schema version 2 (v0.8.0+) changed the documents table:

- `chunk_index` column added — 0 for the first (or only) passage of a document, incrementing for each subsequent passage.
- The unique key changed from `(post_id)` to `(post_id, chunk_index)`. The migration drops the old key explicitly, because `dbDelta` adds keys but never removes them.
- The stored passage column was widened so a full chunk can be held for snippet generation.
- The migration is idempotent and runs on the first admin page load after updating; no data is deleted.

17.5 File layout

```

mbr-intelligent-site-assistant/
|-- mbr-intelligent-site-assistant.php    Main plugin bootstrap + update checker
|-- readme.txt
|-- uninstall.php                        Full-cleanup routine
|-- assets/
|   |-- css/chat-widget.css              Widget styling + theme presets + glass
|   |-- js/chat-widget.js                Widget controller
|-- data/
|   |-- stopwords-en.php                 English stopword list
|-- plugin-update-checker/              Self-hosted update library
`-- includes/
    |-- class-mbr-isa-activator.php       Creates tables, seeds options, migrations
    |-- class-mbr-isa-deactivator.php     Deactivation hooks
    |-- class-mbr-isa-tokeniser.php       Clean / split / stem
    |-- class-mbr-isa-bm25.php            BM25 scoring
    |-- class-mbr-isa-chunker.php         Passage chunking (v0.8.0+)
    |-- class-mbr-isa-pdf-extractor.php   Pure-PHP PDF text extraction
    |-- class-mbr-isa-indexer.php         Index build, search, field weights
    |-- class-mbr-isa-synonyms.php        Synonym expansion
    |-- class-mbr-isa-intents.php         Intent matching
    |-- class-mbr-isa-responder.php       Confidence, messages, snippets
    |-- class-mbr-isa-query-handler.php   Orchestrates the pipeline
    |-- class-mbr-isa-rate-limiter.php    Per-IP throttling
    |-- class-mbr-isa-rest.php            REST route registration
    |-- class-mbr-isa-frontend.php        Shortcode + floating widget render
    |-- class-mbr-isa-cli.php             WP-CLI commands (v0.8.1+)
    |-- class-mbr-isa-admin-intents.php   Admin UI for intents
    |-- class-mbr-isa-admin-synonyms.php  Admin UI for synonyms
    |-- class-mbr-isa-admin-theme.php     Admin UI for appearance
    `-- class-mbr-isa.php                 Singleton orchestrator + diagnostic page

```

17.6 JavaScript integration

The widget exposes a configuration object at `window.mbrAisa`, populated by `wp_localize_script`:

```

window.mbrAisa = {
  restUrl: 'https://example.com/wp-json/mbr-isa/v1/ask',
  feedbackUrl: 'https://example.com/wp-json/mbr-isa/v1/feedback',
  nonce: '...',
  strings: { sending: 'Searching...', networkErr: '...', ... }
};

```

For manual re-initialisation (for example, after an AJAX page swap in a headless theme) the widget exposes `window.MbrAisaChat.init()`.

Note. The JavaScript-side identifiers still use the camelCase `mbrAisa` form, retained from an earlier version of the plugin for backward compatibility with any custom front-end integration code. All PHP-side identifiers are `mbr_isa`.

18. Support and version information

Plugin version	0.8.1
Author	Robert Palmer
Distribution	littlewebshack.com
Updates	Self-hosted — manifest on GitHub, package on littlewebshack.com
Licence	GPL-2.0-or-later
Text domain	mbr-isa
Database schema	2
Requires	WordPress 5.8+, PHP 7.4+
WP-CLI	wp mbr-isa reindex · wp mbr-isa status

This plugin is free and open source. If it has saved you time, a donation at buymeacoffee.com/robertpalmer is very welcome.

For bug reports, feature suggestions, or support questions, please get in touch through littlewebshack.com or via the author's contact channels.